STM32 / FREERTOS TRAINING

Semaphores · Deadlocks · Priority Inversion

01 Semaphores

A semaphore is a kernel object with an integer counter. Two atomic operations — **Take** (P / wait / decrement) and **Give** (V / signal / increment) — are the only way to read or change it. The scheduler blocks a task that calls Take on a zero-count semaphore and unblocks it when another context calls Give.

FreeRTOS provides four semaphore variants. Choosing the wrong one is one of the most common embedded synchronisation bugs.

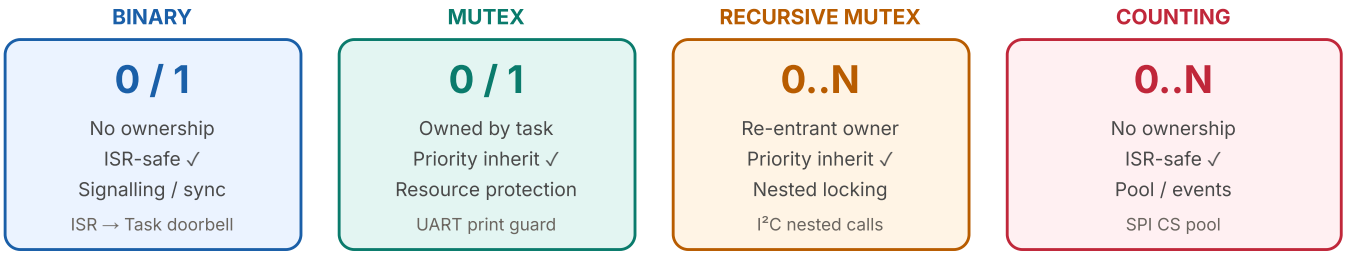


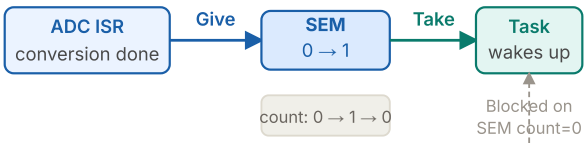
Figure 1 — Four FreeRTOS semaphore variants at a glance

`xSemaphoreCreateBinary()`

Counter is either **0** (taken) or **1** (available). There is no concept of an owner — any task or ISR can Give it.

ISR-SAFE NO PRIORITY INHERIT

Primary use: signalling — an ISR fires and a task wakes to process the event.



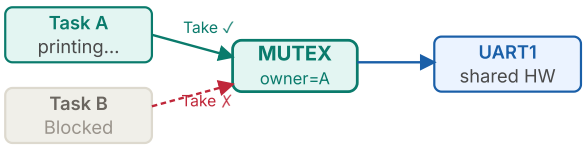
Binary semaphore as ISR→Task doorbell

`xSemaphoreCreateMutex()`

Counter is 0 or 1, but the kernel tracks which task holds it. A task **must** Give back the mutex it Took — any other task Giving it is a bug.

NOT ISR-SAFE PRIORITY INHERIT

Primary use: mutual exclusion — protecting a shared peripheral (UART, SPI, I²C).



Mutex protecting shared UART; Task B blocked until A Gives

`xSemaphoreCreateRecursiveMutex()`

The holding task can Take the same mutex multiple times without deadlocking. A nesting counter tracks

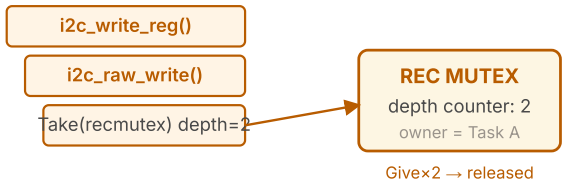
`xSemaphoreCreateCounting(max, init)`

Counter ranges from 0 to a configured maximum. No ownership. ISR-safe with `FromISR` variants.

depth; the mutex is only released when Give is called once for each Take.

NOT ISR-SAFE PRIORITY INHERIT

Use case: a function that locks an I²C bus and calls a helper that also tries to lock the same bus.

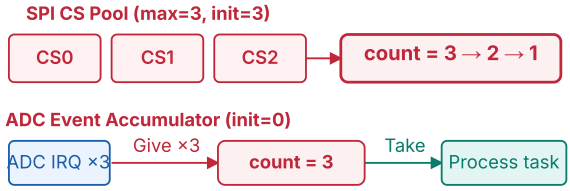


Recursive mutex — nesting depth=2, released only after two Give calls

Tasks taking the semaphore consume a slot; tasks giving it return one.

ISR-SAFE NO PRIORITY INHERIT

Two modes: resource pool (init=max, e.g. 3 SPI CS lines) or event accumulator (init=0, counting pending IRQs).



Counting semaphore — pool and event-accumulator modes

FreeRTOS Semaphore API

Function	Direction	ISR variant	Notes
`xSemaphoreCreateBinary()`	Create	—	Starts at count Give before Tak
`xSemaphoreCreateMutex()`	Create	—	Starts available priority inherita
`xSemaphoreCreateRecursiveMutex()`	Create	—	Must pair with Recursive Give/ API
`xSemaphoreCreateCounting(max, init)`	Create	—	Pool: init=max; Events: init=0
`xSemaphoreTake(sem, ticks)`	Consume	`xSemaphoreTakeFromISR()`	Returns pdTRU success
`xSemaphoreGive(sem)`	Produce	`xSemaphoreGiveFromISR()`	Must call portYIELD_FRO in ISR
`xSemaphoreTakeRecursive()`	Consume	—	Recursive mute
`xSemaphoreGiveRecursive()`	Produce	—	Recursive mute
`vSemaphoreDelete(sem)`	Destroy	—	Never delete wi waiting tasks

Side-by-Side Comparison

Property	Binary	Mutex	Recursive Mutex	Counting
Count range	0–1	0–1	0–N	0–N
Ownership	None	Single task	Single task (nested)	None
ISR safe	✅	❌	❌	✅
Priority inheritance	❌	✅	✅	❌
Primary purpose	Signalling	Mutual exclusion	Re-entrant exclusion	Pool / accumulate
Deadlock risk	Low	Yes (unmatched Take)	Lower (same task re-enters)	Low
FreeRTOS API	`CreateBinary`	`CreateMutex`	`CreateRecursiveMutex`	`CreateCounting`

⚠ GOLDEN RULE

Always do your **work outside the lock**. Copy shared data into a local variable inside the critical section, then release the mutex before processing. Never block, delay, or call slow I/O while holding a mutex.

```
xSemaphoreTake(mutex, portMAX_DELAY);
local_copy = shared_data;          /* copy only — fast */
xSemaphoreGive(mutex);
process(local_copy);               /* work outside — slow is fine */
```

02 Deadlocks

A deadlock is a situation where two or more tasks are each waiting for a resource held by another, so none can ever proceed. The system is stuck permanently — no timeout, no progress.

The Four Coffman Conditions

All four must hold simultaneously for a deadlock to exist. Preventing any one of them prevents the deadlock.

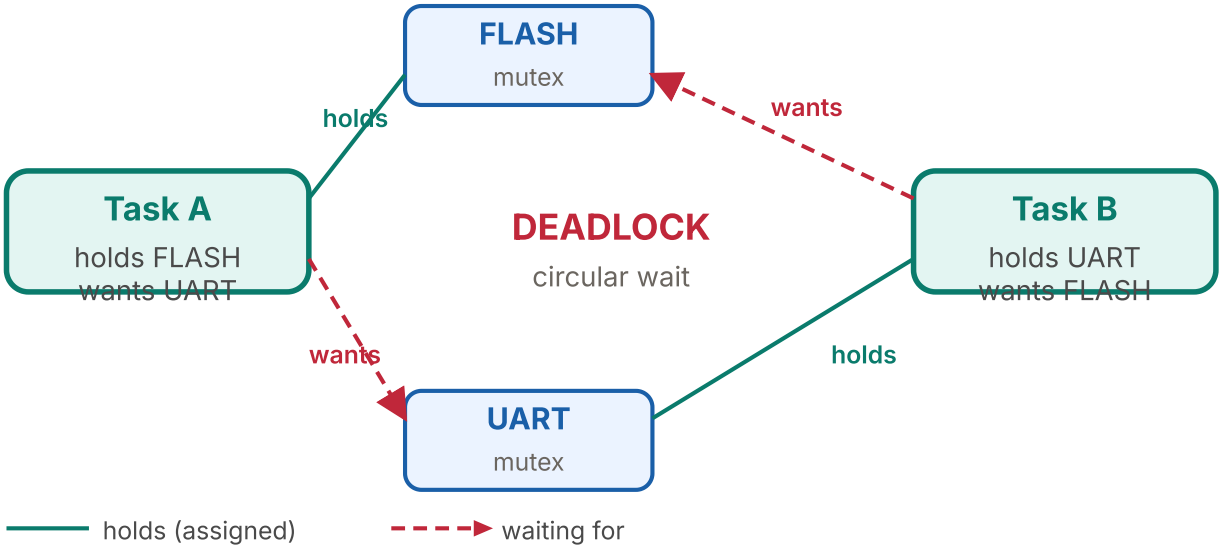


Figure 2 — Circular wait: Task A ↔ Task B holding and waiting for each other's resource

1. **Mutual Exclusion** — the resource can only be held by one task at a time (a mutex, by definition). Cannot be eliminated for truly exclusive hardware.
2. **Hold and Wait** — a task holds at least one resource while waiting to acquire another. Task A holds FLASH mutex and waits for UART mutex.
3. **No Preemption** — resources cannot be forcibly taken away; a task must release them voluntarily. The kernel never revokes a mutex from its owner.
4. **Circular Wait** — a chain of tasks exists where each is waiting for a resource held by the next: A waits for B's resource, B waits for A's resource.

Two-Mutex Deadlock: Step by Step

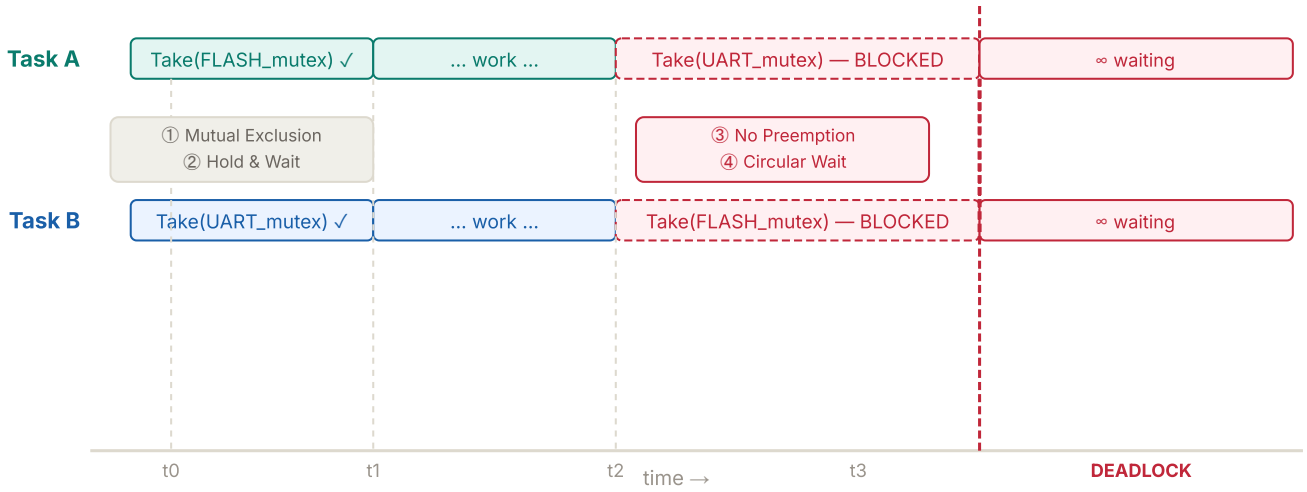


Figure 3 — Two-task, two-mutex deadlock timeline

```
/* Task A: takes FLASH first, then UART */
void TaskA(void *p) {
    for (;;) {
        xSemaphoreTake(flashMutex, portMAX_DELAY);  /* ← holds FLASH */
        vTaskDelay(pdMS_TO_TICKS(10));
        xSemaphoreTake(uartMutex, portMAX_DELAY);   /* ← BLOCKS if B holds UART */
        /* ... work ... */
        xSemaphoreGive(uartMutex);
        xSemaphoreGive(flashMutex);
    }
}

/* Task B: takes UART first, then FLASH  ← OPPOSITE ORDER → deadlock! */
void TaskB(void *p) {
    for (;;) {
        xSemaphoreTake(uartMutex, portMAX_DELAY);   /* ← holds UART */
        vTaskDelay(pdMS_TO_TICKS(10));
        xSemaphoreTake(flashMutex, portMAX_DELAY);  /* ← BLOCKS if A holds FLASH */
        /* ... work ... */
        xSemaphoreGive(flashMutex);
        xSemaphoreGive(uartMutex);
    }
}
```

Prevention Strategies

🔢 **Lock Ordering** (most practical) — assign a global numeric order to all mutexes and always acquire them in the same order. Breaks the circular-wait condition. Both tasks Take FLASH before UART.

```
#define ORDER_FLASH 1
#define ORDER_UART  2
/* Always Take lower-numbered first */
xSemaphoreTake(flashMutex, portMAX_DELAY);
xSemaphoreTake(uartMutex,  portMAX_DELAY);
```

⏱️ **Try-with-Timeout** — use a finite timeout instead of `portMAX_DELAY`. If Take fails, release everything already held and retry later. Breaks hold-and-wait.

```
if (xSemaphoreTake(uartMutex, pdMS_TO_TICKS(100)) ≠ pdTRUE) {
    xSemaphoreGive(flashMutex);   /* release what we hold */
    vTaskDelay(pdMS_TO_TICKS(10));
    continue;                     /* retry outer loop */
}
```

- **Atomic Acquisition** — take all needed resources in one OS call or commit to none (Banker's algorithm). Heavy in practice, rarely used in embedded.

Detection & Recovery

DETECTION

A watchdog timer fed by each task is the simplest deadlock detector in embedded systems. If any task starves its watchdog feed, the system resets.

```
void TaskA(void *p) {
    for (;;) {
        /* ... work ... */
        HAL_IWDG_Refresh(&hiwdg); /* feed */
    }
}
```

RECOVERY

- **Terminate and clean up** — kill one task, manually Give its mutexes, then re-create it
- **Timeout + rollback** — tasks periodically time out and reset their state
- **System reset** — safest in hard real-time: watchdog triggers a full reboot

03 Starvation

Starvation is when a lower-priority task can *theoretically* run but *practically never does* because higher-priority tasks perpetually consume the resource it needs. Unlike a deadlock — where nothing moves — a starved system continues executing, but the low-priority task makes no progress.

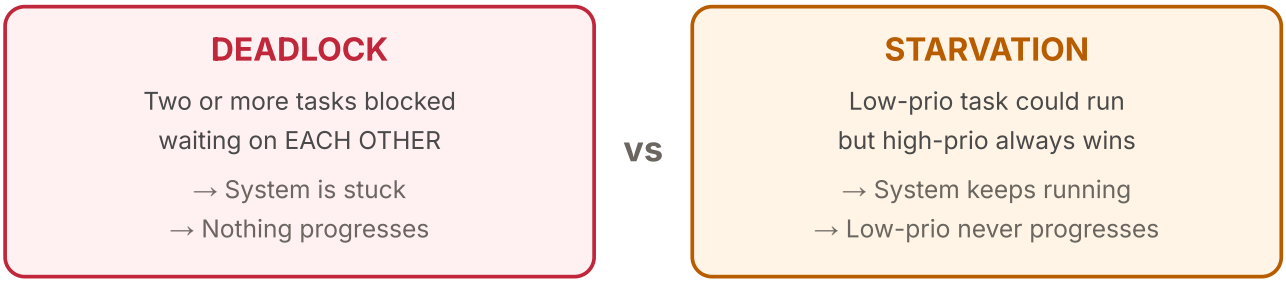


Figure 4 — Deadlock vs Starvation

Starvation Example: Mutex Monopoly

Task HIGH holds a shared mutex for 490 ms out of every 500 ms. Task LOW only gets a 10 ms window per cycle. In practice, the scheduler never catches LOW in that narrow gap.

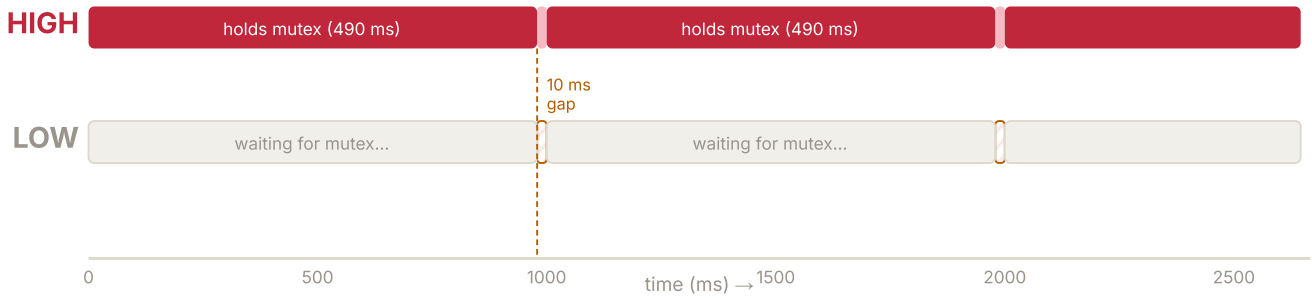


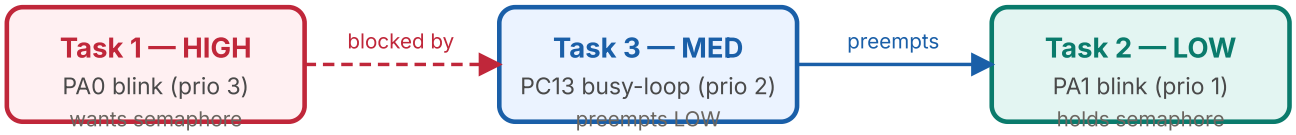
Figure 5 — HIGH task monopolises mutex; LOW task starves (never wins the 10 ms window)

FIX: SHORTER CRITICAL SECTIONS

Hold mutexes for the minimum possible time. Copy data, release, then process. For persistent starvation in FreeRTOS, time-slice with `configUSE_TIME_SLICING = 1` among equal-priority tasks, or limit high-priority task run time to open slots for lower-priority work.

04 Priority Inversion

Priority inversion occurs when a **high-priority task is indirectly blocked by a medium-priority task**, even though it should always preempt medium-priority work. This violates the fundamental guarantee of a priority-preemptive scheduler.



Result: HIGH blocked while MED runs!

Effective priority of HIGH = priority of LOW (inversion)

Binary semaphore used → NO automatic priority inheritance

Figure 6 — Three-task priority inversion setup

Gantt Chart — 3000 ms Timeline

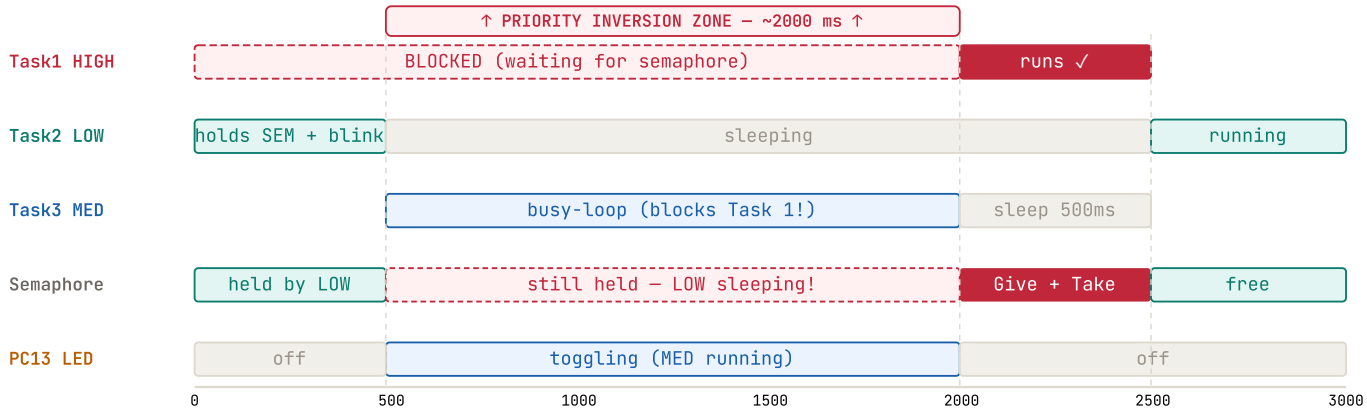


Figure 7 — Priority inversion: Task 1 (HIGH) blocked ~2000 ms while Task 3 (MED) runs freely

Real-World: Mars Pathfinder (1997)

🚀 HISTORICAL BUG

NASA's Sojourner rover suffered repeated system resets on Mars due to priority inversion. An **information bus task (HIGH)** tried to lock a shared bus mutex held by a **meteorological data task (LOW)**. A medium-priority communications task preempted the LOW task, causing HIGH to miss its deadline. The VxWorks watchdog detected the missed deadline and issued a system reset.

The fix was simple: **enable priority inheritance on the mutex** — a runtime config flag that had been left off. The lesson: always use mutexes (not binary semaphores) for resource protection, and enable priority inheritance.

Fix: Priority Inheritance via Mutex

Replace the binary semaphore with a **mutex**. When HIGH-priority Task 1 blocks on the mutex held by LOW-priority Task 2, the scheduler automatically raises Task 2's priority to match Task 1's. This prevents MED-priority Task 3 from preempting Task 2, so Task 2 finishes quickly and Task 1 unblocks.

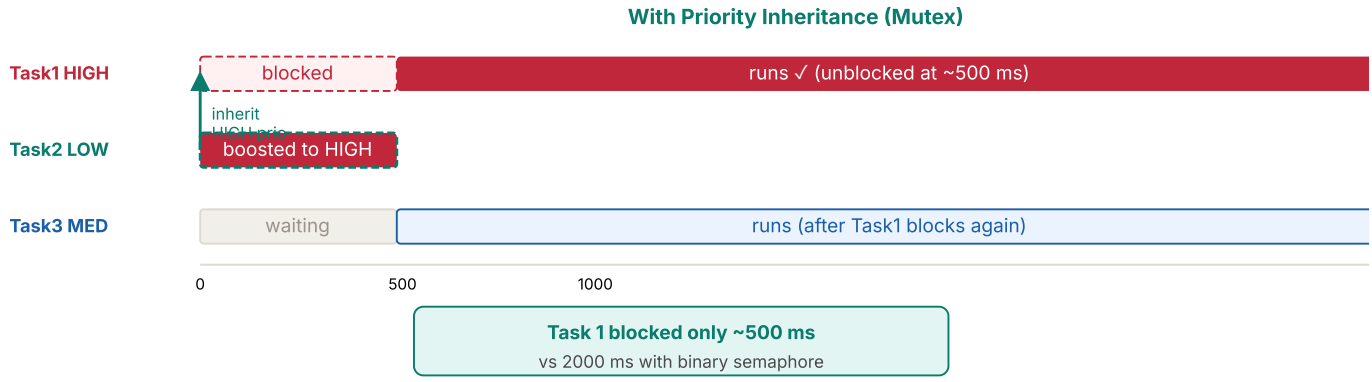


Figure 8 — With mutex: LOW boosted to HIGH priority, MED cannot preempt, Task 1 unblocks ~500 ms

5-STEP PRIORITY INHERITANCE MECHANISM

1. Task2 LOW (prio=1) calls `xSemaphoreTake(mutex)` and acquires it. Starts its critical section.
2. Task1 HIGH (prio=3) wakes and calls `xSemaphoreTake(mutex)` . Mutex is held — Task1 blocks.
3. **Kernel raises Task2's priority to prio=3** (same as the blocked Task1) — this is the inheritance step.
4. Task3 MED (prio=2) tries to preempt but cannot — Task2 now has effective prio=3 and runs to completion.
5. Task2 calls `xSemaphoreGive(mutex)` . Kernel **restores** Task2 to prio=1. Task1 unblocks and runs immediately.

Demo Project Code (Binary Semaphore — shows inversion)

⚙ KEY DESIGN CHOICES

The demo deliberately uses `xSemaphoreCreateBinary()` (not mutex) so that priority inheritance is **disabled** and the inversion manifests. Swap to `xSemaphoreCreateMutex()` to fix it. Task3's busy-wait uses `vApplicationTickHook` + a tick counter to avoid itself being blocked.

```
/* priority-inversion/main.c — simplified */

static SemaphoreHandle_t g_sem;          /* binary — no inheritance! */
static volatile uint32_t g_ticks = 0;

/* Task 1 — HIGH priority (3): blink PA0, waits for semaphore */
void Task1(void *p) {
    for (;;) {
        xSemaphoreTake(g_sem, portMAX_DELAY);   /* blocks when LOW holds it */
        GPIOA->ODR ^= (1u << 0);                /* PA0 toggle */
        vTaskDelay(pdMS_TO_TICKS(500));
        xSemaphoreGive(g_sem);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

/* Task 2 — LOW priority (1): blink PA1, holds semaphore during blink */
void Task2(void *p) {
    for (;;) {
        xSemaphoreTake(g_sem, portMAX_DELAY);
        GPIOA->ODR ^= (1u << 1);                /* PA1 toggle */
        vTaskDelay(pdMS_TO_TICKS(500));         /* holds sem while sleeping! */
        xSemaphoreGive(g_sem);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

/* Task 3 — MED priority (2): busy-loop for 5×500ms, sleep 500ms */
void Task3(void *p) {
    for (;;) {
        uint32_t start = g_ticks;
        while ((g_ticks - start) < 2500u)       /* 5 × 500ms busy */
            ;
        GPIOC->ODR ^= (1u << 13);               /* PC13 toggle */
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

/* Fix: change this line to observe the difference */
g_sem = xSemaphoreCreateBinary();    /* shows inversion  */
/* g_sem = xSemaphoreCreateMutex(); */ /* fixes inversion */
```

✅ SUMMARY: BINARY SEMAPHORE VS MUTEX FOR RESOURCES

	Binary Semaphore	Mutex
Priority inheritance	❌ None	✅ Automatic
Priority inversion risk	High	Mitigated
Ownership tracking	❌	✅
ISR safe	✅	❌
Use for resource protection	❌ Don't	✅ Always